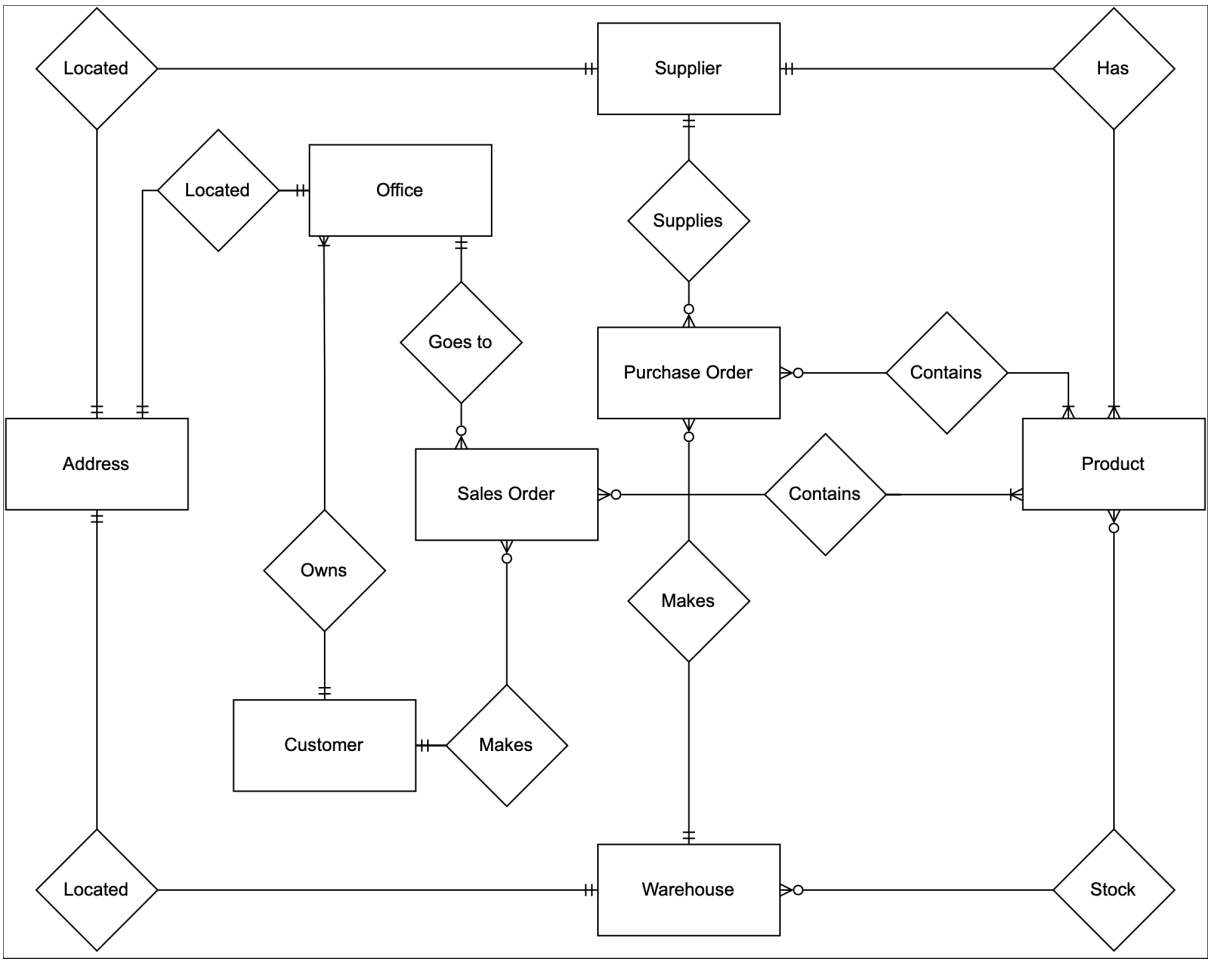
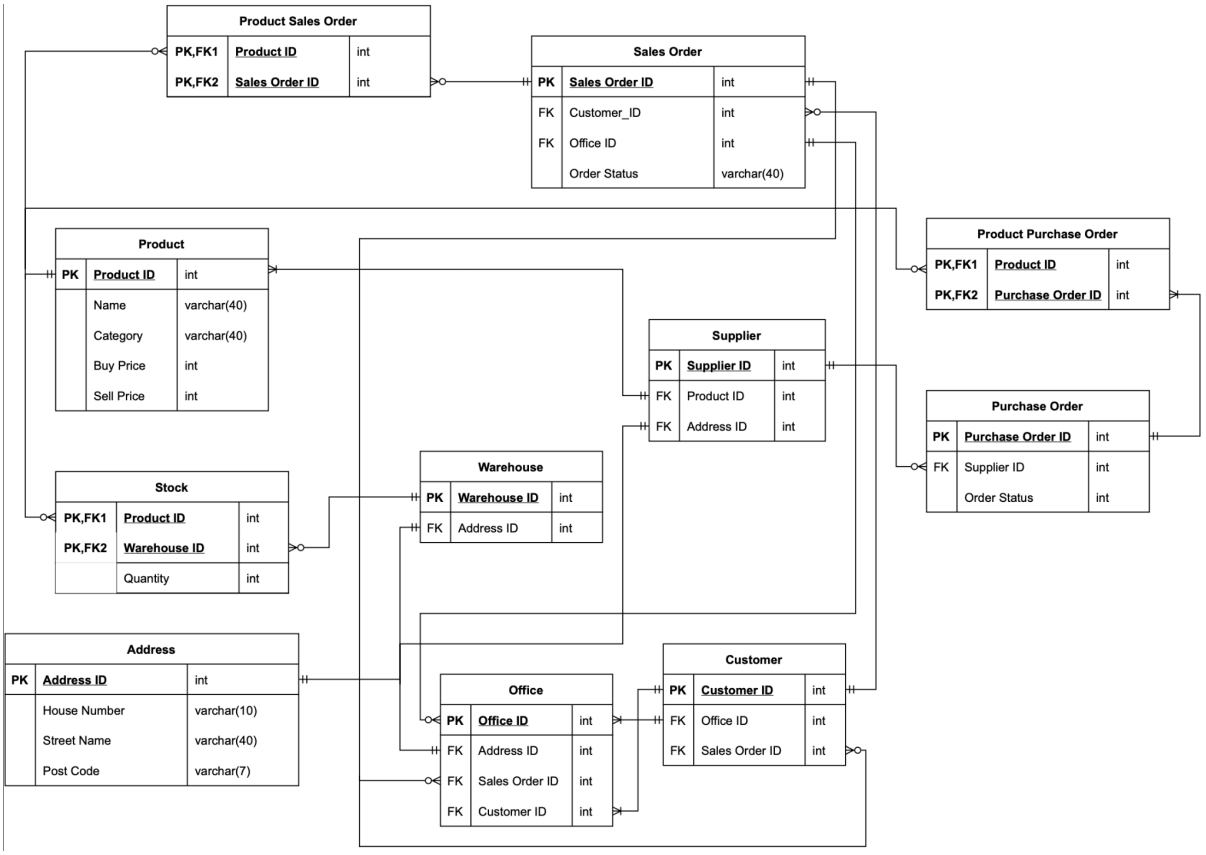

Located
Supplier
Has
Located
Office
Supplies
Goes to
Purchase Order
Contains
Address
Sales Order
Contains
Product
Owns
Makes
Customer
Makes
Located
Warehouse
Stock


Product Sales Order
PK,FK1 Product ID int
PK,FK2 Sales Order ID int
Sales Order
PK Sales Order ID int
FK Customer_ID int
FK Office ID int
Order Status varchar(40)
Product
PK Product ID int
Name varchar(40)
Category varchar(40)
Buy Price int
Sell Price int
Product Purchase Order
PK,FK1 Product ID int
PK,FK2 Purchase Order ID int
Supplier
PK Supplier ID int
FK Product ID int
FK Address ID int
Purchase Order
PK Purchase Order ID int
FK Supplier ID int
Order Status int
Warehouse
PK Warehouse ID int
FK Address ID int
Stock
PK,FK1 Product ID int
PK,FK2 Warehouse ID int
Quantity int
Address
PK Address ID int
House Number varchar(10)
Street Name varchar(40)
Post Code varchar(7)
Office
PK Office ID int
FK Address ID int
FK Sales Order ID int
FK Customer ID int
Customer
PK Customer ID int
FK Office ID int
FK Sales Order ID int

Database Model With Annotations

For this assignment I have created both a conceptual and physical model diagram for a database, based upon a given scenario. The database models convey visual information to communicate the linkage and cardinality between the different entities within the database.

The Process

Entities

I initially began to create the conceptual model for the database by identifying the entities within the brief scenario. Before adding the entities to the database model, I checked that all entities were atomic, to comply with First Normal Form. After adding all entities, I began to add the relationships between entities specified in the requirements. I then went through the database and added relationship assumptions and cardinality to the relations.

Tables

To create the physical model database, I initially added all entities within the conceptual diagram as tables, adding attributes to each of the entities after creating the tables. For the relationships marked in my conceptual diagram as many to many, I added a junction table between the connected entities.

Primary Keys

I then added primary keys for each of the entities. I chose to use int variables as ID's for the primary keys of the entities. Using int variables is good practice in a database as they are easy for a computer to increment when information is added and easier for the computer to find from a query.

Foreign Keys

For all one to many relationships, I added the primary key from the one side as a foreign key on the many side. For all one to one relationships, I identified a parent table and added a foreign key for all child entities relating to that table. For the many to many relationships, I added primary keys that are foreign keys from each of the connected entities to the junction tables.

Relationship Reasoning

As specified in the database requirements, the Supplier, Warehouse and Office each have a unique address, so there is a direct link between each of these entities from the address entity. Each address is specified to be unique, so the cardinality of the linkage is shown with one and exactly one.
The address entity in the physical model has three atomic varchar attributes specifying specific details for an address.

By the requirements, the cardinality of the linkage between the customer and office entities is conveyed, wherein a customer can own one or more offices. My assumption of the relation between the office and customer is shown, where the office is owned by one and exactly one customer.

The customer entity contains two foreign keys to keep track of both the office it is related to and the sales order it could be related to.

Customers can order multiple products as shown by the zero to many relation between the customer and sales order entity. A sales order must have one and only one customer when added to the database. I made the assumption that a sales order must be able to track which office the order goes to for delivery. This is shown in the database that an office is related to zero or more of the sales entity and the sales order must be related to one and exactly one office. A sales order must contain one or more products to be valid. A product can have zero to many sales orders.
Many to many relationships should not exist, as they can create errors within a database.
I placed a junction table between these two entities in the physical model, where the table contains two primary keys made up of the foreign keys from the related entities. The junction table then acts as an interface between the two entities, allowing the two entities to communicate with each other without query issues.

The warehouse entity is shown with a many to many relationship between the warehouse and the product. I used a junction table in the physical diagram that contains two primary keys, that are foreign keys from the related entities. I added an additional variable to track the quantity of a stocked product. The relationship is shown as optional, because I made the assumption that a warehouse can exist within the database without having stock. This may happen when the company wishes to add a new warehouse to their database.

A warehouse can purchase multiple products from a supplier. This is shown in the diagram where the warehouse is related to zero to many purchase orders. The purchase order entity is related to the warehouse such that the relationship from the warehouse to the purchase order is zero to many. A purchase order must belong to a warehouse as this is where the companies store their stock. The supplier supplies the purchase order for the warehouse, where the supplier can have zero to many purchase orders and a purchase order must belong to one and exactly one supplier. A purchase order contains multiple products and a product can be in many purchase orders. I placed a junction table in between the entities to allow the related entities access to one another.

Checking Normal Forms

Normal forms are a way to improve data integrity and reduce redundant data within a database. The database complies to the first normal form if there are no multi-valued attributes. In other words, all attributes must be atomic, where each attribute is unique. I went through and checked that this was true for each entity.
It complies to the second normal form, if it is in the first normal form, and if none of the entities have a functional dependency on another.
I then checked that the database complies with the third normal form, removing all transitive dependencies within the database, checking that no attribute is dependent upon a non-key relation.